Videojuegos

GRADO EN INGENIERÍA MULTIMEDIA

Sesión 06.1

Planificación y Programación ECS

2026-27

¿Verdadero o falso?

✋ **Mano alzada**

1.
2.
3.
4.

¿Verdadero o falso?

✋ **Mano alzada**

1. Una rutina tiene un solo `ret`, al final
2.
3.
4.

¿Verdadero o falso?

✋ **Mano alzada**

1. Una rutina tiene un solo `ret`, al final
2. `ret` sabe de qué rutina sale
3.
4.

¿Verdadero o falso?

✋ **Mano alzada**

1. Una rutina tiene un solo `ret`, al final
2. `ret` sabe de qué rutina sale
3. `call` y `push` usan la misma pila
4.

¿Verdadero o falso?

✋ **Mano alzada**

1. Una rutina tiene un solo `ret`, al final
2. `ret` sabe de qué rutina sale
3. `call` y `push` usan la misma pila
4. El ensamblador sabe dónde acaba cada rutina

1. `call` y `ret`

Paso 1: **`call`** apunta la vuelta en la pila y salta

Programa

```
          . . .
▶ $0150   call rutina
  $0153   ld b, 1
          . . .
```

Rutina

```
  rutina:
  $235A   ld a, 5
          . . .
  $2368   ret
```

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	4A	07	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFE

1. `call` y `ret`

Paso 2: la vuelta ($0153) en la pila y **`SP -= 2`**

Programa

```
        . . .
$0150   call rutina
$0153   ld b, 1          vuelta
        . . .
```

Rutina

```
rutina:
▶ $235A  ld a, 5
        . . .
$2368   ret
```

`vuelta = $0153`

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	53	01	B3	00

◀ la pila crece hacia direcciones menores

`SP = $FFFC`

1. `call` y `ret`

Paso 3: **`ret`** saca la vuelta de la pila y salta

Programa

```
        . . .
$0150   call rutina
$0153   ld b, 1          vuelta
        . . .
```

Rutina

```
rutina:
$235A   ld a, 5
        . . .
▶ $2368   ret
```

vuelta = $0153

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFC

1. call y ret

Paso 4: PC=$0153 y SP+=2. La RAM se ***descarta***

Programa

```
        . . .
  $0150  call rutina
▶ $0153  ld b, 1
        . . .
```

Rutina

```
  rutina:
  $235A  ld a, 5
        . . .
  $2368  ret
```

vuelta descartada (no se borra)

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFE

2. Simular `call`

Sin la instrucción

2. Simular `call`

Sin la instrucción

Rutina

```
✓  $235A  ld a, 5
▶  $235C  call otra_rutina  equivale a ↓
          ld hl, .vuelta
          push hl
          ld hl, otra_rutina
          jp hl
          .vuelta:
```

Otra rutina

```
otra_rutina:
$2400  ld c, 7
$240A  ret
```

CPU

PC	$235C	HL	$7B42

vuelta
$0153

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFC

2. Simular `call`

Paso 1: dentro de **`rutina`**

Rutina

```
✓ $235A  ld a, 5
▶ $235C  ld hl, .vuelta
  $235F  push hl
  $2360  ld hl, otra_rutina
  $2363  jp hl
  $2364  .vuelta:
```

Otra rutina

```
  otra_rutina:
  $2400  ld c, 7
  $240A  ret
```

CPU

PC	$235C	HL	$7B42

vuelta
$0153

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFC

2. Simular `call`

Paso 2: **`HL = $2354`** (dirección retorno)

Rutina

```
✓ $235A  ld a, 5
✓ $235C  ld hl, .vuelta
▶ $235F  push hl
  $2360  ld hl, otra_rutina
  $2363  jp hl
  $2364  .vuelta:          vuelta
```

Otra rutina

```
  otra_rutina:
  $2400  ld c, 7
  $240A  ret
```

CPU

PC	$235F
HL	$2364

antes: $7B42

vuelta $0153

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFC

2. Simular `call`

Paso 3: Retorno en la pila, **`SP -= 2`**

Rutina

```
✓ $235A  ld a, 5
✓ $235C  ld hl, .vuelta
✓ $235F  push hl
▶ $2360  ld hl, otra_rutina
  $2363  jp hl
  $2364  .vuelta:          vuelta
```

Otra rutina

```
  otra_rutina:
  $2400  ld c, 7
  $240A  ret
```

CPU

PC	$2360
HL	$2364

antes: $7B42

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	64	23	53	01	B3	00

vuelta $2364 — vuelta $0153

◀ la pila crece hacia direcciones menores

SP = $FFFA

2. Simular `call`

Paso 4: **`HL = $2400`** para saltar ahí

Rutina

```
✓ $235A  ld a, 5
✓ $235C  ld hl, .vuelta
✓ $235F  push hl
✓ $2360  ld hl, otra_rutina
▶ $2363  jp hl
  $2364  .vuelta:          vuelta
```

Otra rutina

```
  otra_rutina:             destino
  $2400  ld c, 7
  $240A  ret
```

CPU

PC	$2363

HL	$2400

antes: $7B42

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	64	23	53	01	B3	00

vuelta $2364 — vuelta $0153

◀ la pila crece hacia direcciones menores

SP = $FFFA

2. Simular `call`

Paso 5: PC = HL. Igual que **`call`**

Rutina

```
✓ $235A  ld a, 5
✓ $235C  ld hl, .vuelta
✓ $235F  push hl
✓ $2360  ld hl, otra_rutina
✓ $2363  jp hl
  $2364  .vuelta:          vuelta
```

Otra rutina

```
  otra_rutina:             destino
▶ $2400  ld c, 7
  $240A  ret
```

CPU

PC `$2400` HL `$2400`

antes: $7B42

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
											vuelta $2364		vuelta $0153			
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	64	23	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFA

2. Simular `call`

Diferencia: HL modificado. **`call`** no toca registros

Rutina

```
✓ $235A  ld a, 5
✓ $235C  ld hl, .vuelta
✓ $235F  push hl
✓ $2360  ld hl, otra_rutina
✓ $2363  jp hl
  $2364  .vuelta:          vuelta
```

Otra rutina

```
  otra_rutina:             destino
▶ $2400  ld c, 7
  $240A  ret
```

CPU

PC	$2400
HL	$2400

¡pisado! antes: $7B42

vuelta $2364 — vuelta $0153

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	64	23	53	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFA

3. Rutina con `if-else`

`jr`/`jp` solo cambian PC **_la pila no se toca_**



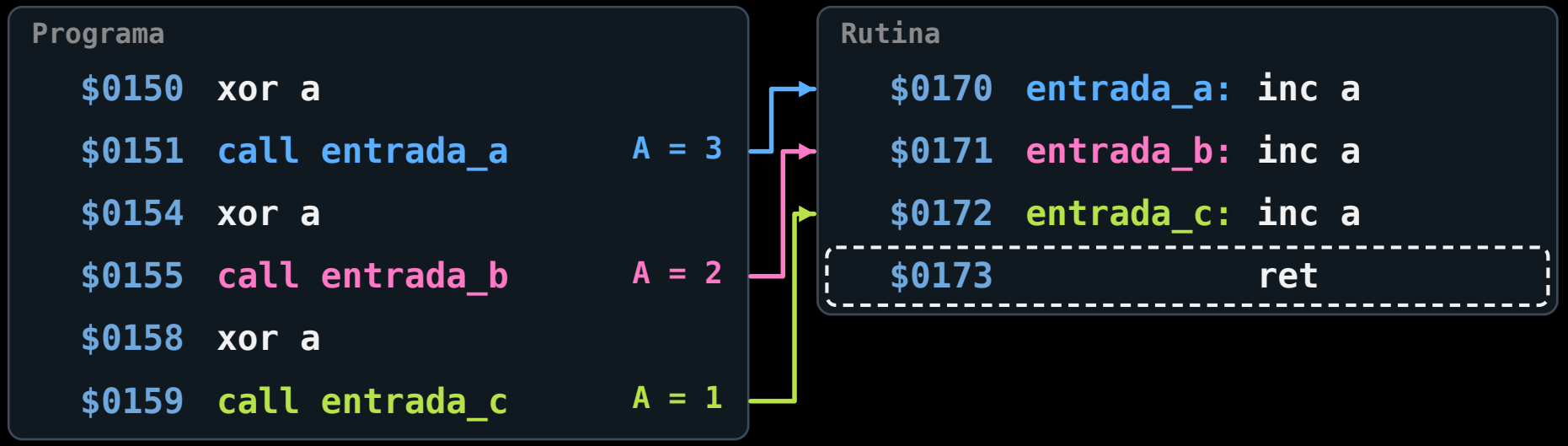
4. Una etiqueta es un número

¿Cuántas "funciones" hay aquí?

```
Rutina
   $0170  entrada_a: inc a
   $0171  entrada_b: inc a
   $0172  entrada_c: inc a
   $0173             ret
```

4. Una etiqueta es un número

Tres llamadas, un solo `ret`



5. `ret` no es de nadie

`ret` no sabe de dónde vino. Salta a ***SP**



6. call_hl

Paso 1: **HL** ya vale el destino.

Programa

```
✓ $0150  ld hl, rutina_x
▶ $0153  call call_hl
  $0156  ld b, 1
         . . .
```

CPU

PC	$0153	HL	$0300

call_hl

```
  $0040  call_hl:   jp hl
```

rutina_x

```
  rutina_x:                    destino
  $0300  ld a, 9
         . . .
  $030A  ret
```

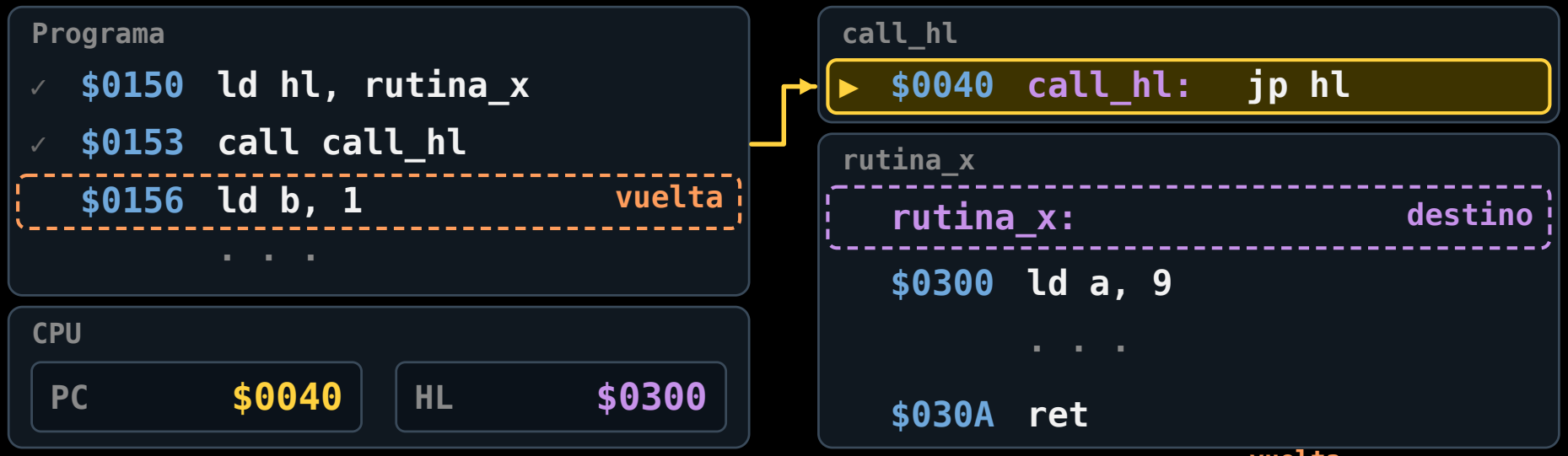
Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	4A	07	B3	00

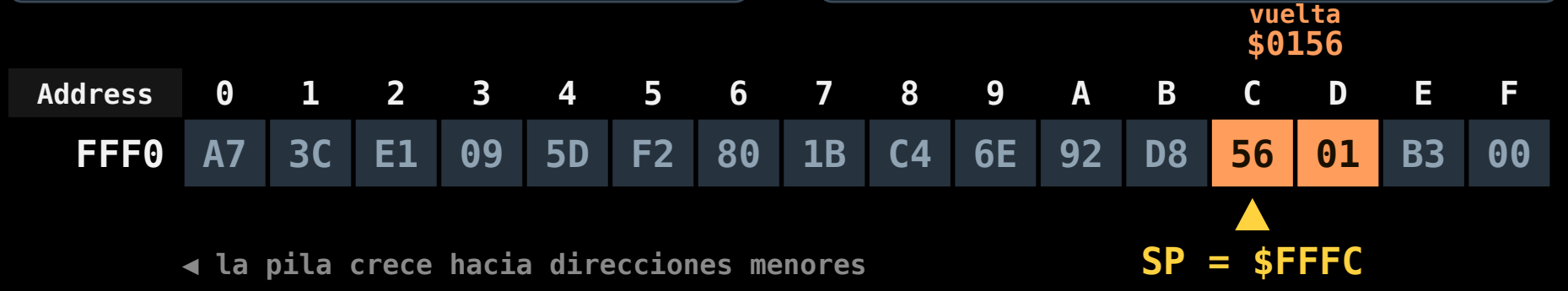
◀ la pila crece hacia direcciones menores

SP = $FFFE

6. call_hl

Paso 2: call guarda retorno ($0156) y salta





6. call_hl

Paso 3: jp hl **PC = HL** salta a la rutina



6. call_hl

Paso 4: llegamos al `ret`

Programa

✓ $0150 ld hl, rutina_x
✓ $0153 call call_hl
$0156 ld b, 1 vuelta
. . .

CPU

PC $030A
HL $0300

call_hl

✓ $0040 call_hl: jp hl

rutina_x

rutina_x: destino
✓ $0300 ld a, 9
. . .
▶ $030A ret

vuelta
$0156

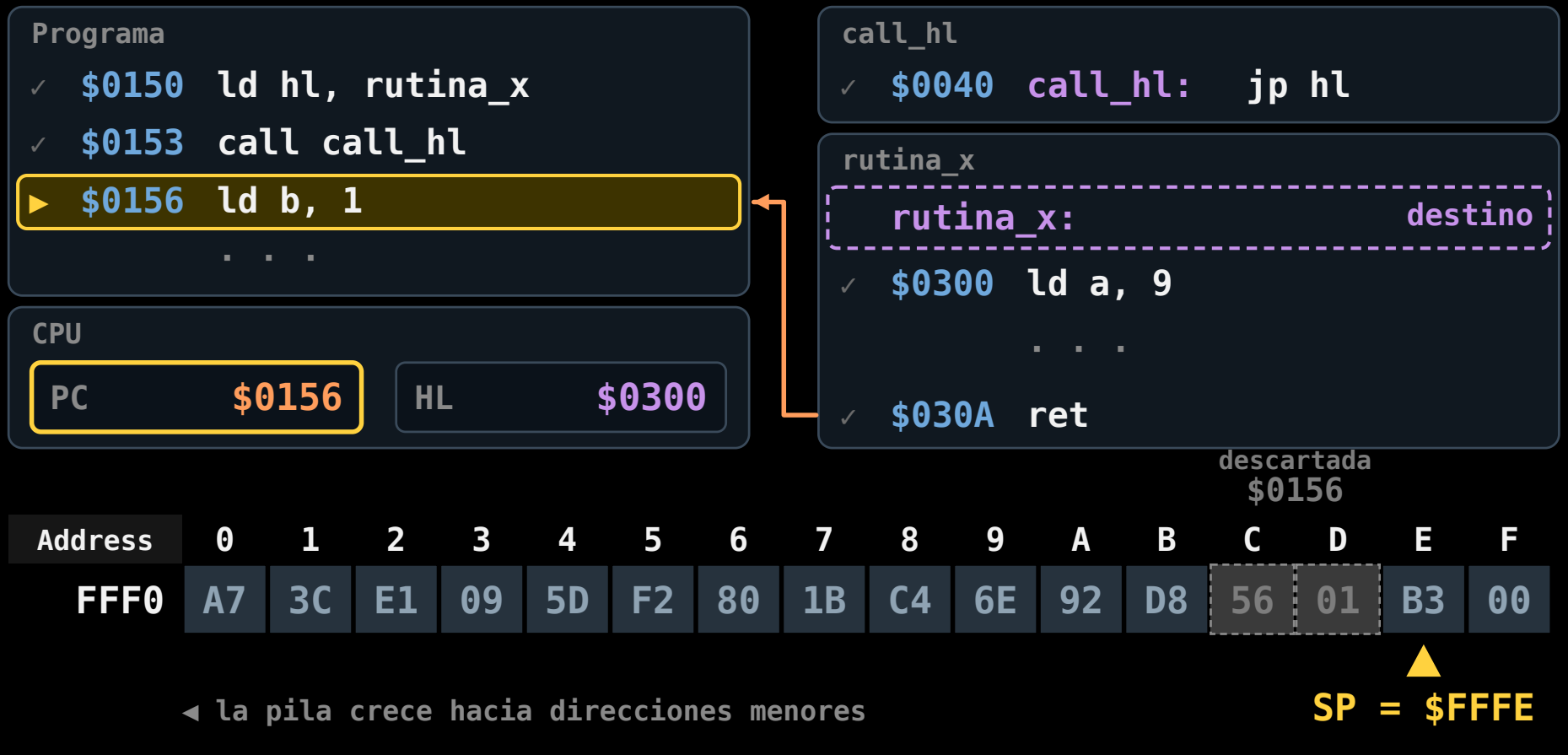
Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	92	D8	56	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFC

6. call_hl

Paso 5: ret -> PC=$0156. Vuelve tras **call**



6. call_hl

Ventaja: Sin **push** ni registro

Sin call_hl

```
$0153  ld bc, .vuelta     pisa BC
$0156  push bc                push
$0157  jp hl
$0158  .vuelta:
```

Con call_hl

```
$0153  call call_hl       sin push
          . . .
el jp hl se comparte: 1 byte, una sola vez
```

	Sin call_hl	Con call_hl
Registros pisados	BC	ninguno
Bytes por llamada	5	3
Ciclos de reloj	32	28

7. Una sola pila

Paso 1: push hl guarda HL en la pila

Programa

```
✓ $0153  call call_hl
  $0156  ld b, 1          vuelta
         . . .
```

rutina_x

```
✓ $0300  push hl
▶ $0301  ld a, 9
  $0303  ret
```

Otro sitio

```
  $4D20  ld a, [de]
  $4D21  inc de
```

CPU

PC	$0301	HL	$4D20

HL $4D20 — vuelta $0156

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
FFF0	A7	3C	E1	09	5D	F2	80	1B	C4	6E	20	4D	56	01	B3	00

◀ la pila crece hacia direcciones menores

SP = $FFFA

7. Una sola pila

Paso 2: `ret` saca lo primero **PC=$4D20$**

Programa

```
✓ $0153  call call_hl
  $0156  ld b, 1          vuelta
         . . .
```

Otro sitio

```
  $4D20  ld a, [de]
  $4D21  inc de
```

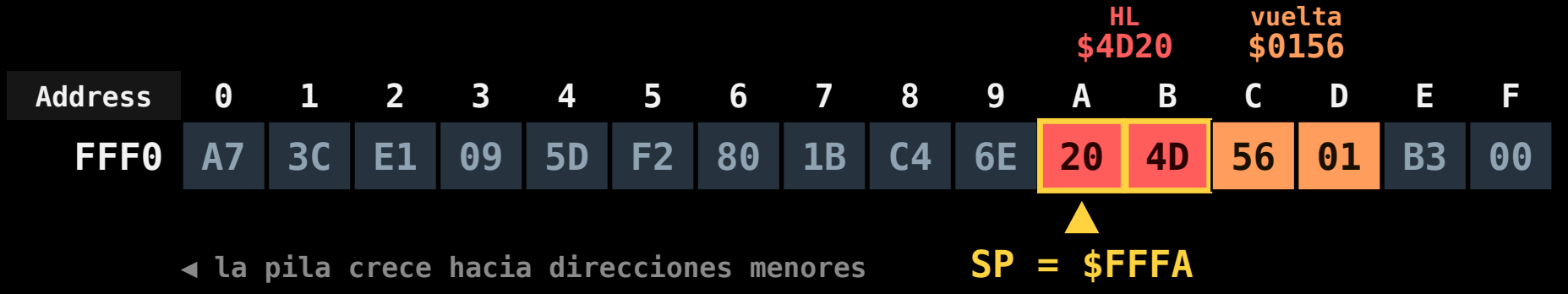
rutina_x

```
✓ $0300  push hl
✓ $0301  ld a, 9
▶ $0303  ret
```

CPU

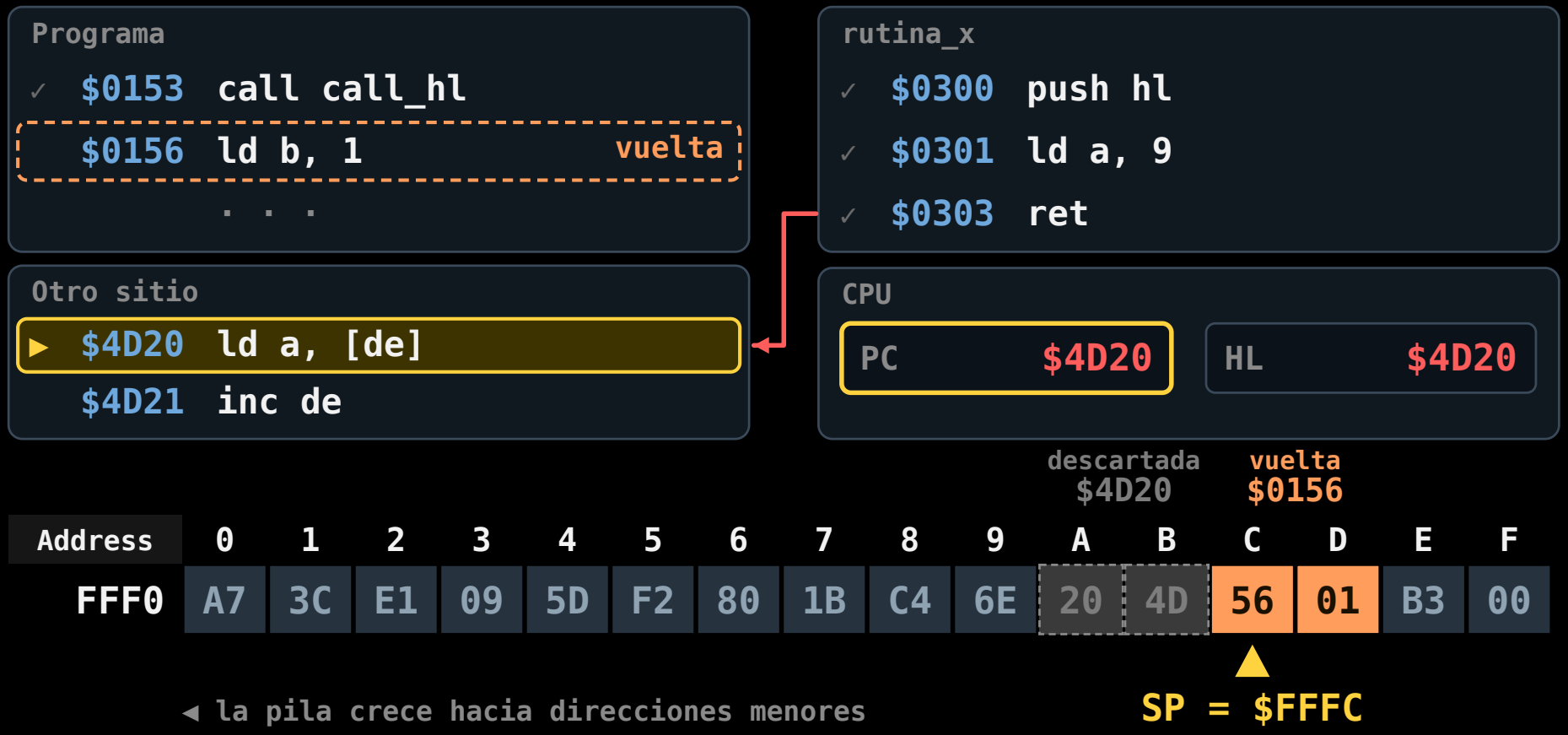
PC	$0303	HL	$4D20

`ret: PC ← $4D20`



7. Una sola pila

Paso 3: **PC = $4D20**. Comportamiento indefinido



De nuevo: ¿V o F?

-
-
-
-
-
-
-
-

De nuevo: ¿V o F?

- Una rutina tiene un solo `ret`, al final
-
-
-
-
-
-
-
-

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*
- **`ret` sabe de qué rutina sale**
-
-
-
-
-
-

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*
- **`ret` sabe de qué rutina sale**
- 🔴FALSO *'saca' lo que haya en la pila (PC=*SP)*
-
-
-
-

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*
- **`ret` sabe de qué rutina sale**
- 🔴FALSO *'saca' lo que haya en la pila (PC=*SP)*
- **`call` y push usan la misma pila**
-
-
-

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*
- **`ret` sabe de qué rutina sale**
- 🔴FALSO *'saca' lo que haya en la pila (PC=*SP)*
- **`call` y `push` usan la misma pila**
- 🟢VERDAD *por eso cada `push` necesita su `pop`*
-
-

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*
- **`ret` sabe de qué rutina sale**
- 🔴FALSO *'saca' lo que haya en la pila (PC=*SP)*
- **`call` y `push` usan la misma pila**
- 🟢VERDAD *por eso cada `push` necesita su `pop`*
- **Ensamblador sabe dónde acaba una `rutina`**
-

De nuevo: ¿V o F?

- **Una rutina tiene un solo `ret`, al final**
- 🔴FALSO *puede haber varios o en otro sitio*
- **`ret` sabe de qué rutina sale**
- 🔴FALSO *'saca' lo que haya en la pila (PC=*SP)*
- **`call` y `push` usan la misma pila**
- 🟢VERDAD *por eso cada `push` necesita su `pop`*
- **Ensamblador sabe dónde acaba una `rutina`**
- 🔴FALSO *las etiquetas son números*

Ilusión de aprendizaje

Ilusión de aprendizaje

😌 Crees aprender

- Ver un vídeo
- Leer una solución
- Repasar

Ilusión de aprendizaje

😌 Crees aprender

- Ver un vídeo
- Leer una solución
- Repasar

💪 Aprendes

- Intentar sin ayuda
- Equivocarte y corregir
- Tolerar incomodidad

Ilusión de aprendizaje

😌 Crees aprender

- Ver un vídeo
- Leer una solución
- Repasar

💪 Aprendes

- Intentar sin ayuda
- Equivocarte y corregir
- Tolerar incomodidad

🔥 ¿Cuándo estoy aprendiendo?

- Ponte a prueba: entender ≠ saber hacer
- Aprendes ***más***, sientes que aprendes ***menos***
- Cuesta esfuerzo, causa frustración

Ilusión de aprendizaje

😌 Crees aprender

- Ver un vídeo
- Leer una solución
- Repasar

💪 Aprendes

- Intentar sin ayuda
- Equivocarte y corregir
- Tolerar incomodidad

🔥 ¿Cuándo estoy aprendiendo?

- Ponte a prueba: entender ≠ saber hacer
- Aprendes ***más***, sientes que aprendes ***menos***
- Cuesta esfuerzo, causa frustración

Deslauriers, L. et al. (2019), Measuring actual learning versus feeling of learning in response to being actively engaged in the classroom, *PNAS* 116(39)

Sin mirar

> *Dibuja cómo funciona `call_hl`*

> *Escribe el código para simularla*

Videojuegos

GRADO EN INGENIERÍA MULTIMEDIA

Sesión 06.1

Planificación y Programación ECS

2026-27